

RBAC for Administrators

Kubernetes Production · Lesson 14

Grant (and revoke) the right access — to humans and to apps

kubernetes.io/docs

What an admin actually does with RBAC

This lesson is **task-oriented** — the 4 things you do for real:

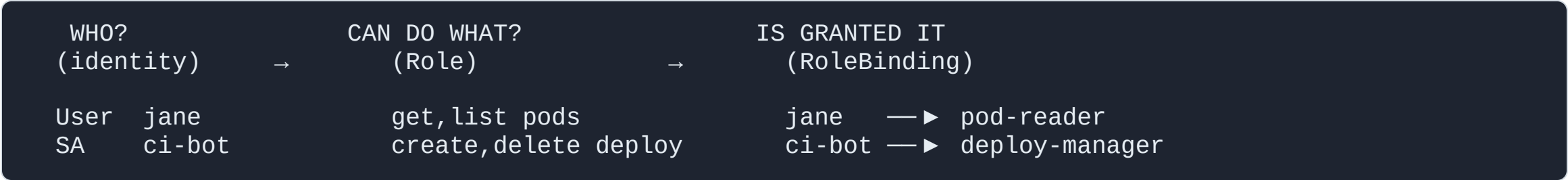
-  **Onboard a human** — give a teammate access to their namespace
-  **Grant an app / CI pipeline access** — via a ServiceAccount
-  **Audit** — answer *"who can do what?"*
-  **Troubleshoot** an `Error ... is forbidden`

A single mental model serves all four. We lay it down first, then apply it.

14.1 The mental model (the only one to remember)

Identity → Role → Binding

An identity can do **nothing** by default. Three building blocks, in this order:



Block	Role	You create it with
Identity	<i>who</i> makes the request	User (certificate) or ServiceAccount
Role	<i>which actions on which resources</i>	<code>kubectl create role</code>
RoleBinding	glues the identity to the Role	<code>kubectl create rolebinding</code>

No RoleBinding = no access. **RBAC has no "deny" rule**: whatever is not granted is refused.

14.2 Identities: human vs app

Two kinds of identity

	 User (human)	 ServiceAccount (app)
For	you, a dev, an external client	a Pod, a CI pipeline, an operator
Object in the cluster	none	a real object (<code>kubectl get sa</code>)
Created by	an external authority (certificate, OIDC)	Kubernetes
Authenticates with	a certificate in the kubeconfig	a token (mounted in the Pod)
Name seen by the API	<code>jane</code>	<code>system:serviceaccount:<ns>:<name></code>

Test question: **"does it run inside a Pod?"** Yes → ServiceAccount.
No (a person) → User.

Creating a User = signing a certificate

There is **no** `kubectl create user`. A human is a **certificate** signed by the cluster; the **CN becomes their name**:

```
openssl genrsa -out jane.key 2048
openssl req -new -key jane.key -out jane.csr -subj "/CN=jane/O=dev-team"
#                                     | O = group
#                                     | CN = user name
```

Submit the CSR to the API, **approve** it, retrieve the signed certificate:

```
kubectl certificate approve jane
kubectl get csr jane -o jsonpath='{.status.certificate}' | base64 -d > jane.crt
```

Full CSR + kubeconfig steps are in the **lab**. Keep the idea:
human = certificate, managed outside Kubernetes.



Creating a ServiceAccount = an object

For an app, no ceremony — one command:

```
kubectl create serviceaccount ci-bot -n dev
```

- The token is **mounted automatically** into Pods that use it:

```
spec:  
  serviceAccountName: ci-bot      # the Pod now acts as ci-bot
```

- From outside (a CI runner), mint a token on demand:

```
kubectl create token ci-bot -n dev --duration=1h
```

Each namespace's **default** SA has **almost no rights** — by design.

The symmetry that simplifies everything

Onboarding a human or an app is **the same recipe**. Only **step 1** changes:

Step	 User jane	 SA ci-bot
1. Identity	a certificate (CN=jane)	<code>kubectl create serviceaccount ci-bot</code>
2. Role	<code>kubectl create role ...</code>	<code>kubectl create role ...</code> <i>(same)</i>
3. Binding	<code>--user=jane</code>	<code>--serviceaccount=dev:ci-bot</code>
4. Verify	<code>--as=jane</code>	<code>--as=system:serviceaccount:dev:ci-bot</code>

Understand **one** of them → you understand both. The RBAC mechanism makes **no** distinction between a human and a robot.

14.3 The Role: what is allowed

Writing a Role

A Role is a list of **rules**: *which verbs on which resources*.

```
kubectl create role pod-reader \
  --verb=get,list,watch --resource=pods -n dev
```

is equivalent to:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
- apiGroups: [""]          # "" = the "core" group: pods, services, configmaps...
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

- Common **verbs**: `get`, `list`, `watch` (read) · `create`, `update`, `patch`, `delete` (write).
- **apiGroups**: `""` for core, `apps` for Deployments, `rbac.authorization.k8s.io` ...

Don't reinvent the wheel: the built-in roles

Kubernetes ships 4 ready-made **ClusterRoles** — reach for them before writing your own:

Built-in ClusterRole	Grants
<code>view</code>	read-only (except Secrets)
<code>edit</code>	create/modify/delete common resources
<code>admin</code>	<code>edit</code> + manage RBAC within the namespace
<code>cluster-admin</code>	everything , everywhere — reserve for <i>break-glass</i>

```
# "jane can edit everything in dev" – without writing a single Role
kubectl create rolebinding jane-edit --clusterrole=edit --user=jane -n dev
```

90% of needs = `view` or `edit` bound in a namespace.

Role vs ClusterRole: just the scope

Same syntax, the only difference is **where** it applies:

In ONE namespace	Across the WHOLE cluster
Role + RoleBinding	ClusterRole + ClusterRoleBinding
"edit in <code>dev</code> "	"read Pods everywhere"
namespaced resources	+ cluster resources (nodes, PV)

Powerful trick: a **RoleBinding** can point at a **ClusterRole**. You write the permission **once** (ClusterRole) and "scope" it to a specific namespace via the RoleBinding.

14.4 The RoleBinding: grant the access

Bind the identity to the Role

The RoleBinding links **subjects** (who) → **roleRef** (which Role):

```
kubectl create rolebinding jane-can-read \
  --role=pod-reader --user=jane -n dev
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: jane-can-read, namespace: dev }
subjects:
- kind: User                # ← or ServiceAccount, or Group
  name: jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

roleRef is **immutable**: to change the target Role, delete and recreate the binding.

14.5 Audit & troubleshoot

"Who can do what?" — kubectl auth can-i

The admin's #1 tool. It answers **yes / no**:

```
# Can I?  
kubectl auth can-i create deployments -n dev  
  
# Can SOMEONE ELSE? (--as = impersonation, admin only)  
kubectl auth can-i list pods --as=jane -n dev  
kubectl auth can-i delete pods \  
    --as=system:serviceaccount:dev:ci-bot -n dev  
  
# EVERYTHING this subject is allowed to do  
kubectl auth can-i --list --as=jane -n dev
```

--as makes you "become" someone for a single command — perfect to **verify an access before claiming it works**.

Troubleshooting a "Forbidden"

The error message **already contains** the whole diagnosis:

```
Error from server (Forbidden): pods is forbidden:
User "system:serviceaccount:dev:ci-bot" cannot list resource
"pods" in API group "" in the namespace "dev"
```

Read it as an RBAC checklist:

The message says...	You check...
User "..."	the right identity ? (expected SA?)
cannot **list**	the verb is missing from the Role
resource "pods" in API group ""	the right resource + apiGroup
namespace "dev"	is the binding in this namespace ?

Reproduce it safely: `kubectl auth can-i list pods --as=... -n dev` → `no`
confirms the gap; add the verb/resource to the Role → `yes` .

Least privilege in 4 reflexes

- Prefer a **Role** in one namespace over a cluster-wide grant.
- **Precise verbs and resources** — avoid `verbs: ["*"]` and `resources: ["*"]`.
- An **app = its own dedicated ServiceAccount** (never the `default` SA).
- Reuse `view` / `edit` before inventing a broad role.

 RBAC stops you from **escalating your own privileges**: you can only grant rights you already hold. Keep `cluster-admin` for emergencies.

Key Takeaways

- **Identity** → **Role** → **RoleBinding**. No binding, no access; there is no "deny" rule.
- **User = certificate** (human, outside the cluster) · **ServiceAccount = object** (app, inside a Pod). Same RBAC recipe afterwards — **only step 1 differs**.
- A **Role** lists `verbs` × `resources`; reuse the built-ins `view` / `edit` / `admin`.
- **Role/RoleBinding** = one namespace; **ClusterRole/ClusterRoleBinding** = the whole cluster.
- Audit and debug with `kubectl auth can-i [--as] [--list]`; the `Forbidden` message **is** your checklist.

End of Lesson 14

Next: Lesson 15 — CRDs & Operators

```
kubectl create role · kubectl create rolebinding · kubectl auth can-i --as
```

Lab: onboard `jane` (User) + `ci-bot` (ServiceAccount), audit, fix a `Forbidden`.